

# Syntaxe podrobněji

Karel Richta a kol.

Katedra technických studií  
Vysoká škola polytechnická v Jihlavě

© Karel Richta, 2020

Základy strukturovaného programování, ZSP  
10/2020, Lekce 5

<https://moodle.vspj.cz/course/view.php?id=200871>



# Doplňky: Preprocesor

## Příklad: *Definice symbolické konstanty*

```
#define MAX 100  
printf("MAX=%d\n",MAX);
```

**MAX=100**

## Příklad: *Definice makra*

```
#define SUM(X,Y) ((X)+(Y))  
printf("SUM(3,7)=%d\n",SUM(3,7));
```

**SUM ( 3 , 7 ) =10**

# Jiná ukázka programu v C

```
/* převodní tabulka Fahrenheit - Celsius:  $C = (5/9)(F-32)$  */
```

```
#include <stdio.h>
```

```
#define POCATEK 0
```

```
#define KONEC 300
```

```
#define KROK 20
```

```
int main(void)
```

```
{
```

```
    int F; float C;
```

```
    F = POCATEK;
```

```
    while (F <= KONEC) {
```

```
        C = (5.0 / 9.0) * (F - 32); /* spočte stupně v C */
```

```
        printf("%4d %6.1f\n", F, C); /* tiskne řádek tabulky */
```

```
        F = F + KROK;          /* posune na další F */
```

```
    }
```

```
    return 0;
```

```
}
```

definice  
symbolických  
konstant (makra  
bez parametrů)



# Preprocesor – další doplňky

## Podmíněný překlad:

```
#if konstantní výraz  
#ifdef jméno  
#ifndef jméno  
#elif konstantní výraz  
#else  
#endif
```

## Příklad:

```
#if MAX > 10  
    Zde je kód pro MAX > 10  
#else  
    Zde je kód pro MAX <= 10  
#endif
```

# Preprocesor – další doplňky

## **Příklad:** *vícenásobné větvení*

```
#if MAX > 10
```

    Zde je kód pro MAX > 10

```
#elif MAX > 5
```

    Zde je kód pro MAX <= 10 a MAX > 5

```
#else
```

    Zde je kód pro MAX <= 5

```
#endif
```

# Preprocesor – další doplňky

**Příklad:** *korektní předefinování makra*

```
#ifdef MAX  
#undef MAX  
#endif  
#define MAX 100
```

# Využití podmíněného překladu

**Příklad:** stráže hlaviček knihoven (umožňují libovolný počet vložení hlavičky)

```
#ifndef __STDIO_H
#define __STDIO_H
...
    tělo knihovny
...
#endif
```

**Alternativa:**

```
#pragma once
```

# Preprocessor – další doplňky

- Preprocesor nabízí některá předdefinovaná makra:
  - `__FILE__` jméno zdrojového souboru,
  - `__LINE__` číslo řádky v aktuálním souboru,
  - `__func__` jméno aktuálně překládané funkce,
  - `__cplusplus` příznak, zda je zdrojový kód kompilován překladačem C nebo C++.
- Kompilátory pak nabízejí předdefinovaná makra specifická pro výrobce/platformu:
  - `WIN32` příznak kompilace pro Windows,
  - `BORLANDC` Borland kompilátory,
  - `DEBUG` příznak kompilace pro ladění,
  - `RELEASE` příznak kompilace pro ostré nasazení.



# Preprocesor – další doplňky

Předdefinovaná makra mohou usnadnit ladění:

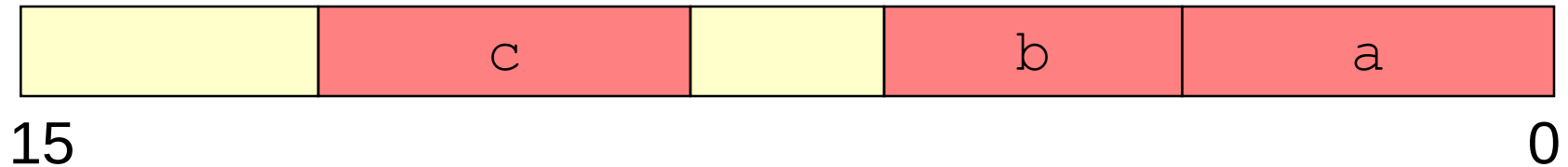
```
int divide(int num, int denom) {  
    if (denom == 0)  
    {  
        fprintf(stderr, "Delení nulou\n"  
            " funkce: %s "  
            " soubor: %s"  
            " radek: %d\n", __func__, __FILE__, __LINE__);  
        return (0);  
    }  
    return (num / denom);  
}
```

# Doplňky k deklaracím

- Bitová pole
- Deklarace sjednocení (**union**)
- Deklarace funkcí
- Konstantní výrazy

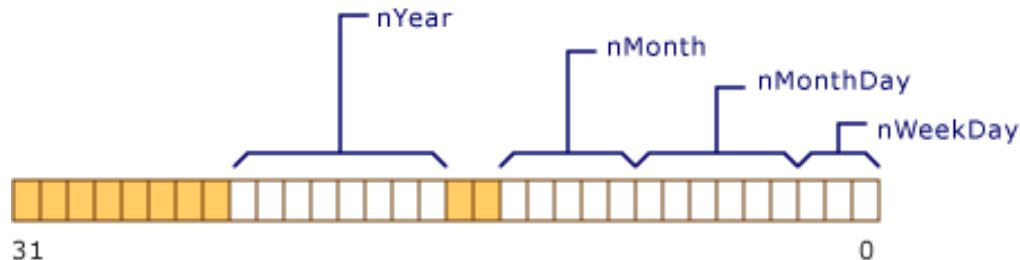
# Bitová pole

```
struct {  
    unsigned a:4;  
    signed b:3;  
    int :2;  
    int c:4;  
}
```



# Příklad na bitová pole

```
// bit_fields1.cpp
// compile with: /LD
struct Date { // reprezentace data na 32 bitech
    unsigned short nWeekDay : 3;    // 0..7   (3 bity)
    unsigned short nMonthDay : 6;   // 0..31  (6 bitů)
    unsigned short nMonth    : 5;   // 0..12  (5 bitů)
    unsigned short           : 2;
    unsigned short nYear     : 8;    // 0..100 (8 bitů)
};
```

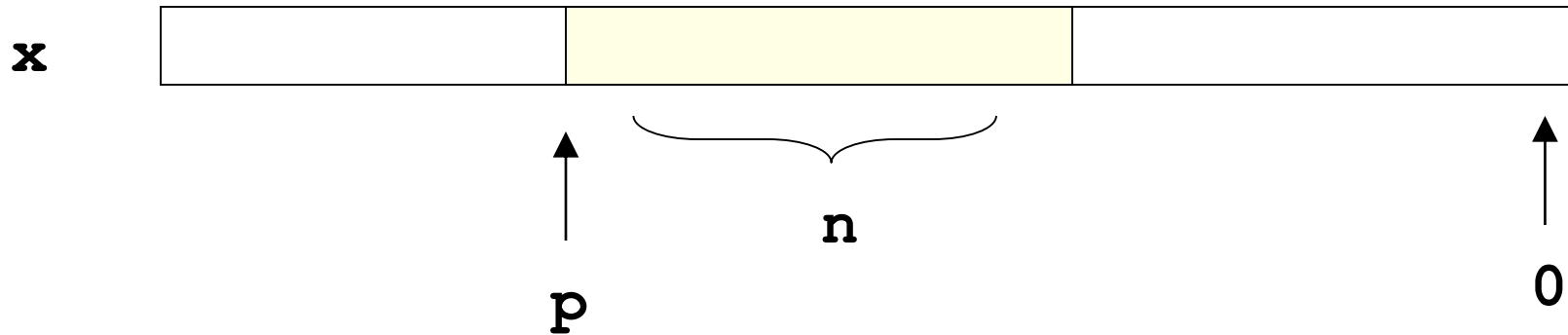


<https://msdn.microsoft.com/cs-cz/library/ewwyfdbbe.aspx>

# Bitové operace

- Operandy jen celočíselné, provádějí se běžné aritmetické konverze
- Unární:
  - ~ negace všech bitů
- Binární:
  - & logický součin všech bitů
  - | logické součet všech bitů
  - ^ logické *xor* všech bitů
  - << posun bitové reprezentace levého operandu vlevo
  - >> posun bitové reprezentace levého operandu vpravo
- U operací posunu pravý operand udává délku posunu v bitech
- Pozor, plete se & a &&, | a || :
  - x = 1; y = 2;
  - x & y == 0
  - x && y == 1

## *Příklad: vyříznutí $n$ bitů z $x$ od pozice $p$*



```
unsigned getbits(unsigned x, unsigned p, unsigned n)
{
    return (x >> (p-n+1)) &
           (~((~0)<<n));
}
```

# Indexace

- Syntaxe:  $X[Y]$ , kde jeden výraz je typu ukazatel na  $T$  a druhý typu  $int$ , výsledek je typu  $T$
- Jméno pole prvků typu  $T$  je konstantní ukazatel na  $T$
- Příklady:  

```
int a[10], i, *p;  
a[1] = 5;  
p = a;  
*(p + 2) = 0;  
p[2] = 0;
```
- Výraz  $X[Y]$  je ekvivalentní s  $*(X + (Y))$
- Žádná kontrola indexů se neprovádí
- Vícerozměrná pole jsou pole polí, indexace v každém rozměru zvlášť  

```
int mat[2][3];  
mat[1][2] = 1;
```
- Pozor:  
mat[1,2] je dovoleno, neznamená však dvojitou indexaci!!

# Aritmetika s ukazateli

- Ukazatelé mohou být operandy sčítání a odčítání
- Dovolené kombinace a typ výsledku:

$T^* + \text{int} \rightarrow T^*$

$T^* - \text{int} \rightarrow T^*$

$T^* - T^* \rightarrow \text{int}$

- Přičtení  $n$  k ukazateli typu  $T^*$  znamená jeho změnu o  $n$ -násobek délky typu  $T$ , podobně odečtení a rozdíl ukazatelů.
- Příklady:

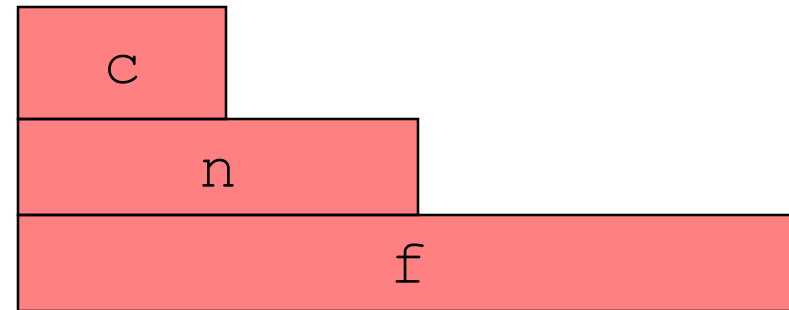
```
int a[10], *p = &a[0];
*(p + 3) = 10;          /* do a[3] se uloží 10 */
/* vynulování pole a */
for (p = &a[0]; p <= &a[9]; p++) *p = 0;
/* nebo */
for (p = &a[0]; p <= &a[9]; *p++ = 0);
/* nebo */
for (p = a; p <= a + 9; *p++ = 0);
```



# Popis typu union

- Syntaxe stejná jako u struktury, místo **struct** je **union**
- Položky se nekladou za sebe, ale přes sebe
- Typ **union** se používá jako variantní záznam
- Příklad:

```
union U {  
    char c;  
    int  n;  
    float f;  
} x;
```



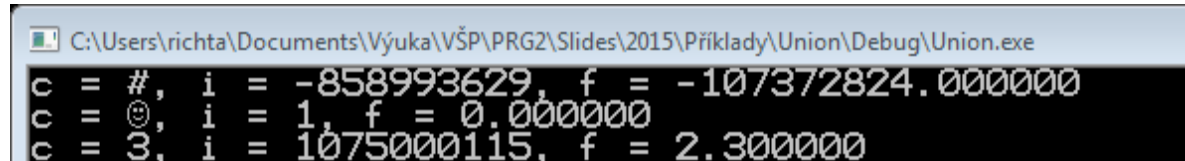
- Inicializuje se dle první položky.  
union U z = 'a';

# Příklad

```
#include <stdio.h>

typedef union {
    char  c;
    int    i;
    float f;
} ZN_INT_FLT;
```

```
int main() {
    ZN_INT_FLT a, *p_a = &a;
    a.c = '#';
    printf("c = %c, i = %d, f = %f \n", a.c, a.i, a.f);
    p_a->i = 1;      /* premaze znak '#' */
    printf("c = %c, i = %d, f = %f \n", a.c, a.i, a.f);
    a.f = 2.3;      /* premaze cislo 1 */
    printf("c = %c, i = %d, f = %f \n", a.c, a.i, a.f);
}
```



```
C:\Users\richta\Documents\Výuka\VŠP\PRG2\Slides\2015\Příklady\Union\Debug\Union.exe
c = #, i = -858993629, f = -107372824.000000
c = ☺, i = 1, f = 0.000000
c = 3, i = 1075000115, f = 2.300000
```

# Selekce

- Zpřístupnění položky struktury, třídy nebo unionu
- Přímá selekce:

***X . položka***      kde *X* je výraz typu **struct** nebo **union**

- Nepřímá selekce (přes ukazatel):

***X -> položka***      kde *X* je výraz typu ukazatel na **struct** nebo **union**

- *X -> položka* je zkratkou za *(\*X) . položka*
- Příklad:

```
struct { int a; char b; } x, *px = &x;
```

```
x.a = 1;
```

```
px -> b = 'a';
```

```
(*px).b = 'a'; /* totéž */
```

# Operátor délky typu

- Zjištění velikosti vnitřní reprezentace objektu nebo typu
- Syntaxe:

***sizeof*** výraz *nebo*

***sizeof*** ( *označení typu* )

- Výsledek v bytech (přesněji nejmenších adresovatelných jednotkách), typu ***size\_t*** (většinou ***unsigned***)
- Výraz se nevyhodnocuje, jména polí se nekonvertují na ukazatele!
- Příklady:

```
int pole[] = {1, 2, 3, 4};
```

```
sizeof(pole) / sizeof(int) /* počet prvků pole */
```

```
sizeof(void *) /* délka vnitřní reprezentace ukazatelů */
```

```
sizeof(T *) /* délka vnitřní reprezentace ukazatelů (na T  
nezáleží) */
```

# Přetypování (casting)

- Explicitní změna typu (většinou u ukazatelů), může znamenat změnu vnitřní reprezentace
- Syntaxe:

**( T ) X**

kde *T* je označení skalárního typu nebo **void** a *X* je výraz skalárního typu

- Příklady:

```
int a, b; float r;  
r = (float)a / b;
```

```
int *p;  
char *q;  
q = p;                /* chyba nebo varovné hlášení */  
q = (char*)p;         /* O.K. */
```

- Přetypování může být složitější, pokud se jedná o objekty, jejichž typ se může měnit (např. ukazatel na nějakou hierarchii tříd). Pak je nutno rozlišovat, zda má být přetypování statické, či dynamické a je nutno používat operátory `static_cast<typ>`, příp. `dynamic_cast<typ>`:

```
q = static_cast<char*>(p);
```

# Ternární podmíněný operátor

- Výsledek výrazu je závislý na hodnotě podmínky
- Syntaxe:

***Podmínka ? Výraz1 : Výraz2***

- kde *Výraz1* a *Výraz2* jsou kompatibilních typů
- Je-li *Podmínka* splněna, je výsledkem hodnota *Výrazu1*, jinak je výsledkem hodnota *Výrazu2*
- Vyhodnocuje se podmínka a pak jen příslušný výraz
- Příklad:

`max = a > b ? a : b;`

`p = r * (x < 0 ? -1 : x == 0 ? 0 : 1);`

# Operátor čárka

- Umožňuje několik výrazu v místě, kde se očekává jeden výraz (nejčastěji v příkazu **for**, viz dále)
- Syntaxe:

***Výraz1 , Výraz2 , ... , Výrazn***

- Výrazy se vyhodnotí v pořadí zleva doprava, výsledkem je hodnota posledního výrazu (hodnoty ostatních výrazů se "zahodí")
- Pozor: může škodit (viz indexace)
- Nejčastější použití v příkazu **for**:

```
for (v = 1, i = n; i > 1; v *= i, i--);
```

# Konstantní výrazy

- Konstantní výraz je výraz vyhodnotitelný v době překladu
- Nesmí obsahovat vedlejší efekty
- Použití:
  - počet prvků pole
  - inicializace statických objektů
  - návěští v přepínači atd.

```
int pole[10];      // 10 je konstanta
int pole[5 + 7];   // součet konstant je konstanta
const int velikost = 20;
int pole[velikost]; // v pořadku, velikost je const
int velikost = 20;
int pole[velikost]; // chyba, velikost není const
```



# Priorita a asociativita operátorů

|                                            |               |
|--------------------------------------------|---------------|
| [ ] ( ) . -> postfix ++ and postfix --     | Zleva-doprava |
| prefix ++ and prefix -- sizeof & * + - ~ ! | Zprava-doleva |
| () - přetypování                           | Zprava-doleva |
| * / %                                      | Zleva-doprava |
| + -                                        | Zleva-doprava |
| << >>                                      | Zleva-doprava |
| < > <= >=                                  | Zleva-doprava |
| == !=                                      | Zleva-doprava |
| &                                          | Zleva-doprava |
| ^                                          | Zleva-doprava |
|                                            | Zleva-doprava |
| &&                                         | Zleva-doprava |
|                                            | Zleva-doprava |
| ? :                                        | Zprava-doleva |
| = *= /= %= += -= <<= >>= &= ^=  =          | Zprava-doleva |
| ,                                          | Zleva-doprava |

# Abstraktní deklarátor

v popisu formálních parametrů v deklaraci funkce, při přetypování, nebo jako parametr operátoru **sizeof** lze vypustit identifikátor, tj. použít **abstraktní deklarátor**

|                           |                                                                                                             |
|---------------------------|-------------------------------------------------------------------------------------------------------------|
| <b>int *</b>              | ukazatel na <b>int</b>                                                                                      |
| <b>char (*) (int)</b>     | ukazatel na funkci z <b>int</b> do <b>char</b>                                                              |
| <b>unsigned *[4]</b>      | pole 4 ukazatelů na <b>unsigned</b>                                                                         |
| <b>int (* (*) () ) ()</b> | ukazatel na funkci, která vrací<br>ukazatel na funkci s libovolným<br>počtem parametrů vracející <b>int</b> |

# Třídy identifikátorů v C

- jména maker - zpracuje preprocesor (nezávislá na ostatních jménech)
- návěští - pozná se **nav**: (lokální ve funkci - musí být jednoznačné ve funkci)
- jména struktur, sjednocení a výčtů (musí být jednoznačná v rámci jednotlivých kategorií)
- jména položek (musí být jednoznačná v rámci struktury nebo sjednocení)
- ostatní jména (proměnné, funkce, typy, konstanty výčtu) musí být v rozsahu platnosti jednoznačná

# Rozsah platnosti identifikátorů

- globální objekty - od místa deklarace do konce zdrojového souboru
- formální parametry - lokální objekty v těle funkce
- formální parametry v deklaraci funkce - do konce deklarace
- lokální objekty - od místa deklarace do konce bloku
- návěští - lokální v těle funkce
- makro - od direktivy **#define** do konce zdrojového souboru nebo direktivy **#undef**

# Doplňky k příkazům

- Strukturované skoky (break, continue)
- Přepínače (switch)
- Příkazy skoku a návratu (goto, return)
- Volání funkcí
- Parametry volané hodnotou

# Strukturované skoky

- Příkaz *break*:

**break ;**

- ukončí bezprostředně nadřazený cyklus nebo přepínač

- Příkaz *continue*:

**continue ;**

- přejde na nové vyhodnocení podmínky cyklu

```
while ( podm1 ) {  
    ...  
    if ( podm2 ) break;  
    ...  
    if ( podm3 ) continue;  
    ...  
}
```

# Příkaz větvení (přepínač)

- Příkaz **switch**:

**switch** ( *výraz* ) *příkaz*

- slouží k rozvětvení výpočtu do několika větví podle hodnoty celočíselného výrazu
- příkazem je téměř vždy složený příkaz
- některé z příkazů ve složeném příkazu jsou označeny návěštími ve tvaru:

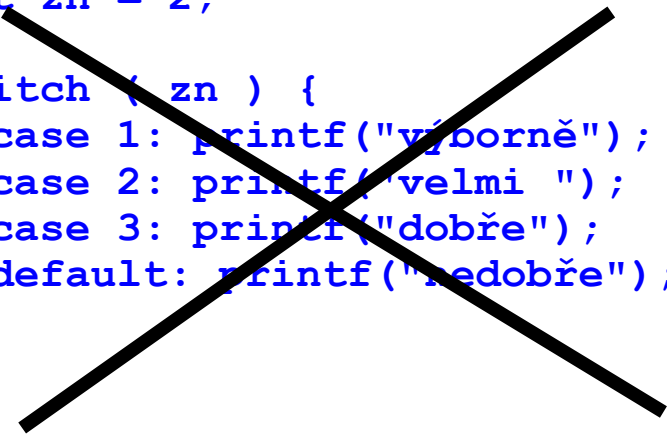
**case** *konst. výraz*:    nebo                    **default**:

- po vyhodnocení výrazu se pokračuje tou variantou, která je označena návěští se stejnou hodnotou (návěští musí být různá)
- návěští **default** pokrývá ostatní hodnoty
- každá větev obvykle končí příkazem **break**

```
switch ( n ) {  
    case 1: printf("*"); break;  
    case 2: printf("***"); break;  
    case 3: printf("****"); break;  
    default: printf("-")  
}
```

## *Příklad: použití přepínače*

```
int zn = 2;  
  
switch ( zn ) {  
    case 1: printf("výborně");  
    case 2: printf("velmi ");  
    case 3: printf("dobře");  
    default: printf("nedobře");  
}
```



**Chybně**

```
int zn = 2;  
  
switch ( zn ) {  
    case 1: printf("výborně");  
            break;  
    case 2: printf("velmi ");  
    case 3: printf("dobře");  
            break;  
    default: printf("nedobře");  
}
```

**Správně**



# Příkaz skoku a návratu

- Příkaz skoku:

**goto** *návěští* ;

- návěští je identifikátor
- použití: výskok z více úrovní vnoření cyklu apod.
- lze skákat dovnitř strukturovaných příkazů - nedoporučuje se

- Příkaz s návěštím:

*návěští* : *příkaz*

- návěští jsou lokální ve funkci

```
for (i = 1; i < 10; i++)
    for (j = 1; j < 20; j++) {
        ... if (podm) goto konec; ...
    }
konec: ;
```

- Příkaz **return**:

**return** *výraz* ;      nebo      **return** ;

- ukončí funkci a definuje návratovou hodnotu

# Deklarace funkce dle ANSI

## *hlavička tělo (= blok)*

- Hlavička má tvar prototypu (profilu) funkce: paměťová třída, typ návratové hodnoty, jméno funkce a seznam deklarací parametrů.
- Každý parametr je deklarován zvlášť:

```
long Power(long x, int y)
{
    long result = 1;
    while (y-- >= 0) result *= x;
    return result;
}
void Dummy(void) { }
```

- Typ návratové hodnoty **void** = funkce nevrací hodnotu (procedura).
- **void** místo seznamu parametrů = funkce bez parametrů.
- Volání funkcí definovaných ANSI stylem se kontroluje a provádějí se případné konverze skutečných parametrů.

# Příklad – použití prototypů

```
// použití funkčních prototypů
#include <stdio.h>

void odd(int x);
void even(int x);

int main() {
    int i;
    do {        printf("Please, enter number (0 to exit): ");
                scanf("%d", i);
                odd(i);
            } while (i != 0);
    return 0;
}

void odd(int x) {
    if ((x % 2) != 0) printf("It is odd.\n"); else even(x);
}

void even(int x) {
    if ((x % 2) == 0) printf("It is even.\n"); else odd(x);
}
```

# Volání funkce

- Syntaxe:

**$F ( X1, X2, \dots Xn )$**

- kde  $F$  je ukazatel na funkci a  $X1, \dots Xn$  jsou výrazy tvořící skutečné parametry volání
- Identifikátor funkce je konstantním ukazatelem na funkci
- Při volání funkce deklarované ANSI stylem se:
  - kontroluje počet parametrů a typy parametrů
  - skutečné parametry se konvertují pomocí roztažení (promotion) a navíc typ **float** se konvertuje na **double**
  - kontroluje přípustnost skutečných parametrů a provádějí případné konverze (podobně jako při přiřazení)
- Pozor: závorky jsou třeba, i když se jedná o funkci bez parametrů!
- V C může existovat v celém programu jen jedna funkce daného jména, v C++ může existovat více funkcí stejného jména, ale musí být rozlišitelné pomocí parametrů (tzv. přetížení identifikátorů funkce)

# *Příklady volání funkcí*

```
long Power(long, int);  
long l; int a, b; float f;  
l = Power(l, 10);  
a = Power(a + 3, b);  
f = Power(f, 3);
```

```
void Swap(int *, int *);  
int x, y;  
Swap(&x, &y);
```

```
void CtiPole(int, int *);  
int pole[10];  
CtiPole(10, pole);
```

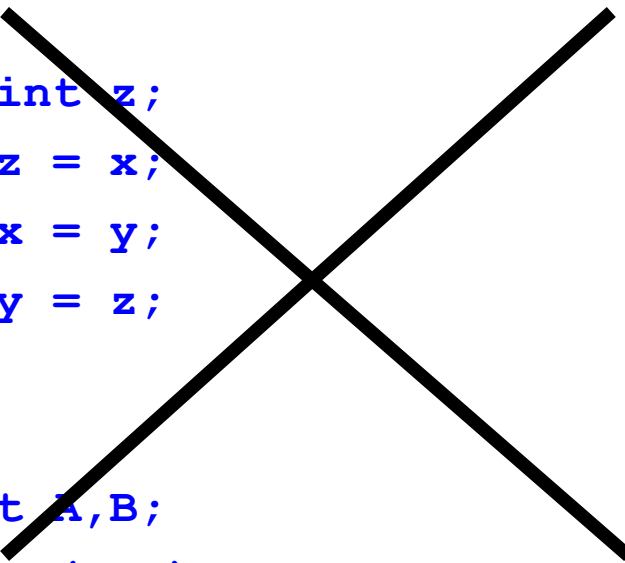
# Parametry funkce volané hodnotou

- Parametry jsou volány hodnotou – to znamená, že jako skutečné parametry můžeme použít libovolný výraz převeditelný na daný typ. Výraz se vyhodnotí a jeho hodnota se uloží do formálního parametru, který funguje jako lokální proměnná.
- Funkce s parametry volanými hodnotou vrací nejvýše jeden výsledek – funkční hodnotu (zpravidla přes zásobník). Pokud vrací void – nevrací nic (je to procedura).
- Pokud chceme, aby funkce vracela více hodnot, musíme využít globální proměnné, nebo použít jako parametry ukazatele (v C++ lze ještě použít parametry volané referencí).
- Přes parametry typu ukazatel můžeme manipulovat s objekty mimo lokální prostor těla funkce. To může vyžadovat určitou disciplinu.

## *Příklad: funkce, která prohodí 2 buňky*

```
void swap(int x, int y)
{
    int z;
    z = x;
    x = y;
    y = z;
}

int A,B;
swap(A,B);
```



**Chybně**

```
void swap(int *x, int *y)
{
    int z;
    z = *x;
    *x = *y;
    *y = z;
}

int A,B;
swap(&A, &B);
```

**Správně**

# Pořadí vyhodnocení parametrů

- Není normou jazyka definováno, implementace to může udělat tak, jak to vyjde efektivněji (pro procesory Intel zpravidla zprava doleva, pro procesory Sparc zleva doprava – BigIndian/LittleIndian).
- Musíme programy psát tak, aby na tom nezáleželo – ve skutečných argumentech nepoužívat operace s vedlejšími efekty.

```
#include <stdio.h>

void foo(int x, int y) {}

int main() {
    foo(sprintf("foo"), sprintf("bar"))
    return 0;
}
```

Ve Windows vytiskne:

barfoo

Na Linuxu:

foobar



# *Cvičení: Pořadí vyhodnocení parametrů*

```
#include <stdio.h>

void f(int x, int y, int z) {}

int p(int x)
{
    printf("Parametr: %d\n", x); return x;
}

int main()
{
    f(p(1), p(2), p(3));
    return 0;
}
```

# Rekurzivní funkce

```
// factorial calculator
#include <stdio.h>

long factorial(long a) {
    if (a > 1)
        return (a * factorial(a - 1));
    else
        return 1;
}

int main() {
    long number = 9;
    printf("%ld ! = %ld\n", number, factorial(number));
    return 0;
}
```

# The End